

预编译

```
1 #define ld long double
2
3 typedef long double ld
4
5 using ld = long double;
```

decltype 用法

`decltype` 用于根据表达式自动推断类型。

`auto` 也是自动类型推导

示例

```
1 int a = 10;
2 decltype(a) b = 20; // b 的类型和 a 一样，都是 int
3
4 double x = 3.14;
5 decltype(x + a) y; // y 的类型是 double
6
7 // 用于函数返回值类型推断
8 template<typename T, typename U>
9 auto add(T t, U u) -> decltype(t + u) {
10     return t + u;
11 }
```

不同const常量

```
1 #define a 1
2 const int b = 1;
3 constexpr int c = 1; #编译期常量，速度更快
4 #错误写法
5 int n;
6 cin>>n;
7 constexpr int c = n;
8 #constexpr是编译期常量，不能在运行期赋值
```

const 成员函数

不能调用非const函数

const也能作为成员函数的重载函数区分（const类优先匹配const函数）

constexpr

~~~~

以下描述中，正确的是？

A、

```
#include <iostream>

constexpr int add(int x, int y)
{
    return x + y;
}

int main()
{
    constexpr int N = add(42, 7);
    int x[N];
    // ...
}
```

代码中这样的用法，在C++中是合法的。因为，在编译期间就可以确定N的值是常量值49。

B、

```
#include <iostream>

int add(int x, int y)
{
    return x + y;
}

int main()
{
    constexpr int N = add(42, 7);

    // ...
}
```

代码中这样的用法，在C++中是不合法的。因为，add()是一个普通函数。而用于constexpr变量N初始化的值必须是一个常量值。

- C、`constexpr int N = 42;`  
这样定义后，N的值是在编译期间就确定的常数。

```
constexpr int get_size() {  
    return 42;  
}
```

- D、`}`  
这样定义后，`get_size()`是一个`constexpr`函数。这个简单代码片段中，函数在编译期间可以确定结果。也就是说，有些计算，在编译期间就可以完成，不再占用执行时间，从而达到提高程序执行效率的目的。

## 赋值

```
1 | int xq = 1;  
2 | int aq(4.5); #可以运行，但是会缺失精度  
3 | int s1{4.5} #编译速度快，但不可运行，要求必须满足类型
```

## 多种类型转换

```
1 | static_cast<int>(5.2);  
2 |
```

## 引用变量

多个引用变量可以绑定到同一个变量

引用 必须初始化

```
1 | int x = 42;  
2 | int &rx = x;  
3 | int &rx2 = x;  
4 |  
5 | #错误写法  
6 | int &r = 42  
7 | #正确写法  
8 | const int &r = 42;  
9 | int&& r = 42;
```

## for循环

```
1 | vector<string> v1{"hi", "add", "xyz"};  
2 |  
3 | for(auto i : v1)  
4 |     #创建副本  
5 |  
6 | for(const auto &i : v1)  
7 |     #引用类型，不加const可以更改值  
8 |     #更有效率
```

# 控制输出

```
1 #include <iomanip>
2
3 cout<<setw(4)<<x<<endl;
4 #设置输出宽度
5 cout<<setfill('#')<<set(4)<<x<<endl;
6 设置输出宽度，不足的用‘#’填充
```

# 内联函数

简单的、频繁调用的函数

如果有 循环 和 switch 语句，则编译器 不同意 设置为内联函数，因为不够简单

# 带有默认值的函数

默认值要在函数 后面， 靠右放置

如果有 函数声明，在函数声明中给 默认值，定义中不要重复给。也可反过来，但 不推荐。

- 一定 不可以同时标明默认参数

# 模板函数

不管是函数还是声明，都要重复定义

```
1  template<typename T>
2  void func(T x)
3  {
4      ....;
5  }
6
7  template<typename T>
8  void func(T x)
9  {
10     ....;
11 }
```

# 类

构造函数可以重载，析构函数不可重载

类函数的实现在 类定义内部，则是 内联函数，否则不是

public:公有

private:私有

protected:

再不定义构造函数的时候，编译器自动生成默认构造函数，如果定义了构造函数，则不自动生成

下面的构造函数可以运行，因为构造的只有默认参数

```

1  X(int x = 4):m{x} {}
2
3
4  X x;

```

```

1  class X {
2  public:
3      // 成员函数
4      X(); #默认构造
5      X(int a_, int b_, int c_); #普通构造
6      X(const X &t); #复制构造
7      X(X &&t); #移动构造 // x2(move(x1)); #移动构造
8
9      ~X(); #析构函数
10
11     void set(int a_, int b_, int c_);
12     int sum() const;
13     int product() const;
14 private:
15     // 成员数据
16     int a, b, c;
17 };
18
19 X::X(int a_, int b_, int c_):a{a_},b{b_},c{c_}
20 {
21     cout<<"1\n";
22 };
23
24 X::X()
25 {
26     cout<<"hrere\n";
27 };
28
29 X::X(const X &t):a{t.a},b{t.b},c{t.c}
30 {
31     cout<<"copy\n";
32 }
33
34 X::~~X()
35 {
36     cout<<"destroy\n";
37 }

```

## 临时对象

可以不定义类直接使用

```

1  X().show();
2  X(42).show();

```

## 初始化内嵌对象

```
1
2 C(int x,int y,int z):obj_A{x},obj_B{y},mc{z} {}
3
4 private:
5     B obj_B;
6     A obj_A;
7     int mc;
```

- 最后调用顺序是：B->A->C
- 按照类定义里声明的顺序构造

运行测试结果：

```
B: constructor called. 2
A: constructor called. 1
C: constructor called. 3
C: destructor called. 3
A: desctructor called. 1
B: destructor called. 2
```

说明：

先按声明顺序（B→A）构造内嵌对象，再构造C类对象；析构逆序。

## 右值引用

`std::move()` 强制转换成右值

右值 不可寻址

左值 可以寻址

```
1 int x =42;
2 int y =x;
3
4 int &r1 = x; #左值
5 int &&r2 = 42;#右值引用
6 int &&r3 = func();#右值引用
7 int &&r4 = std::move(x);
```

原因:

`std::move(x)` 只是把 `x` 强制转换为右值引用类型 (`int&&`) , 并没有转移或修改 `x` 的值。只有在和“移动构造/赋值”配合时, 才会导致资源转移、原变量变空。

在你的代码里:

```
int x = 4;
int &&r = move(x);
cout << r << " " << x << endl; // 输出 4 4
```

`r` 只是 `x` 的别名 (引用) , 两者本质上是同一个变量。

只有像 `string`、`vector` 这种有资源的类型, 配合 `std::move` 才会让原变量变“空”。内置类型 (如 `int`、`double`) 不会有任何变化。

## 类中的const变量

只能通过 构造函数初始化列表 初始化

## 类中的static变量

在 类的外部 需要进行 定义 和 初始化

```
1  #include <iostream>
2  using namespace std;
3  class MyClass {
4  public:
5      static int staticVar; // 声明静态成员变量
6  };
7  // 定义并初始化静态成员变量
8  int MyClass::staticVar = 0;
9  int main() {
10     MyClass obj1, obj2;
11     // 通过类名访问静态变量
12     MyClass::staticVar = 10;
13     cout << "StaticVar (via class): " << MyClass::staticVar << endl;
14     // 通过对象访问静态变量
15     obj1.staticVar = 20;
16     cout << "StaticVar (via object): " << obj2.staticVar << endl;
17     return 0;
18 }
```

## 注意

- 静态成员变量的定义必须在类的外部进行，且只能初始化一次。
- 如果静态成员变量是 `const`，可以在类内直接初始化。
- 静态成员函数不能访问非静态成员，因为它们没有 `this` 指针。

```
1  #include <iostream>
2  using namespace std;
3  class MyClass {
4  public:
5      static const int staticVar;
6  };
7
8  const int MyClass::staticVar = 0;
9  int main() {
10
11      return 0;
12  }
```

## 友元friend

类的友元函数是定义在类外部，但有权访问类的所有私有（private）成员和保护（protected）成员。尽管友元函数的原型有在类的定义中出现过，但是友元函数并不是成员函数。

友元可以是一个函数，该函数被称为友元函数；友元也可以是一个类，该类被称为友元类，在这种情况下，整个类及其所有成员都是友元。

基类的友元函数是派生类的友元函数，但是友元的友元不是友元，没有传递

## 深复制和浅复制

- 浅复制会导致复制版类中的指针也指向原类中指针的地址（指针复制地址），编译器默认的是浅复制
- 深复制完全开辟新的空间，需要自己实现（vector等自己已实现）

## 继承？

```
1  class Y:public std::vector<int>{
2  public:
3      ///using std::vector<int>::vector;
4      Y() = default;
5      Y(int n) : vector<int>(n){}
6      Y(std::initializer_list<int> ls) : vector<int>(ls){}
7
8      void sort(bool flag = true)
9      {
10         if(flag)
11             std::sort(this->begin(),this->end());
12         else
13             std::sort(this->begin(),this->end(),std::greater<int>());
14     }
```



```
15 };
```

- 继承方式(public,private,protected)缺少时, 默认 `private`
- `private` 成员在类外部不能访问
- 其余成员, 权限按小的 (继承方式和基类的权限相比较)

## 注意点

- 构造函数/析构函数不能被继承
- 派生类须自定义构造函数/析构函数
- 若都采用编译器的默认构造函数, 则可不写
- 基类的类型指针和引用可以指向派生类, 但调用的是基类接口 (如果在基类的接口加上`virtual`, 则可调用相应的派生类接口)

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class A
5  {
6      private:
7          int a,b;
8          void func(){cout<<"private function\n";}
9      public:
10         void func1(){cout<<"public function\n";}
11 };
12
13 class B : public A{
14     public:
15         void func1(){cout<<"B public function\n";}
16 };
17
18 int main()
19 {
20     B b;
21     //调用基类的func1,使用作用运算符
22     b.A::func1();
23     //调用自己的func1
24     b.func1();
25 }
```

## 虚基类

在C++中, 虚基类 (Virtual Base Class) 是用来解决特定的继承问题的一种机制。当多个派生类继承自同一个基类, 并且又有一个类继承自这些派生类时, 就会出现一个问题: 最终的派生类将包含多个基类的副本。为了解决这个问题, C++引入了虚基类的概念, 确保在继承层次中基类只有一个唯一的实例。

当声明一个类为虚基类时, 使用关键字`virtual`。这告诉编译器在派生类中只保留一个基类的副本, 即使有多个中间派生类从同一个基类继承。这样, 当有多个路径可以到达同一个基类时, 基类的构造函数只会被调用一次, 避免了多重继承时的二义性问题。

- `virtual public`虚继承方式 (`class A : virtual public B{}`)

```

1  class Base {
2  public:
3      Base(int s) {
4          a = s;
5          cout << "base=" << a << endl;
6      }
7  protected:
8      int a;
9  };
10 class Base1 : virtual public Base {
11 public:
12     Base1(int s, int h) : Base(s) {
13         a += h;
14         cout << "base1=" << a << endl;
15     }
16 };
17 class Base2 : virtual public Base {
18 public:
19     Base2(int s) : Base(s) {
20         a += 20;
21         cout << "base2=" << a << endl;
22     }
23 };
24 class Derived : public Base1, public Base2 {
25 public:
26     Derived(int s, int h, int d) : Base(s), Base1(s, h), Base2(s) {
27         cout << "derived a =" << a + d << endl;
28     }
29 };
30 int main() {
31     Derived obj(5, 8, 9);
32     return 0;
33 }

```

# 多态

---

## 绑定

---

- 动态绑定
- 静态绑定

# 运算符重载

---

下面是不可重载的运算符列表：

- `.`：成员访问运算符
- `.*`, `->*`：成员指针访问运算符
- `::`：域运算符
- `sizeof`：长度运算符
- `?:`：条件运算符
- `#`：预处理符号